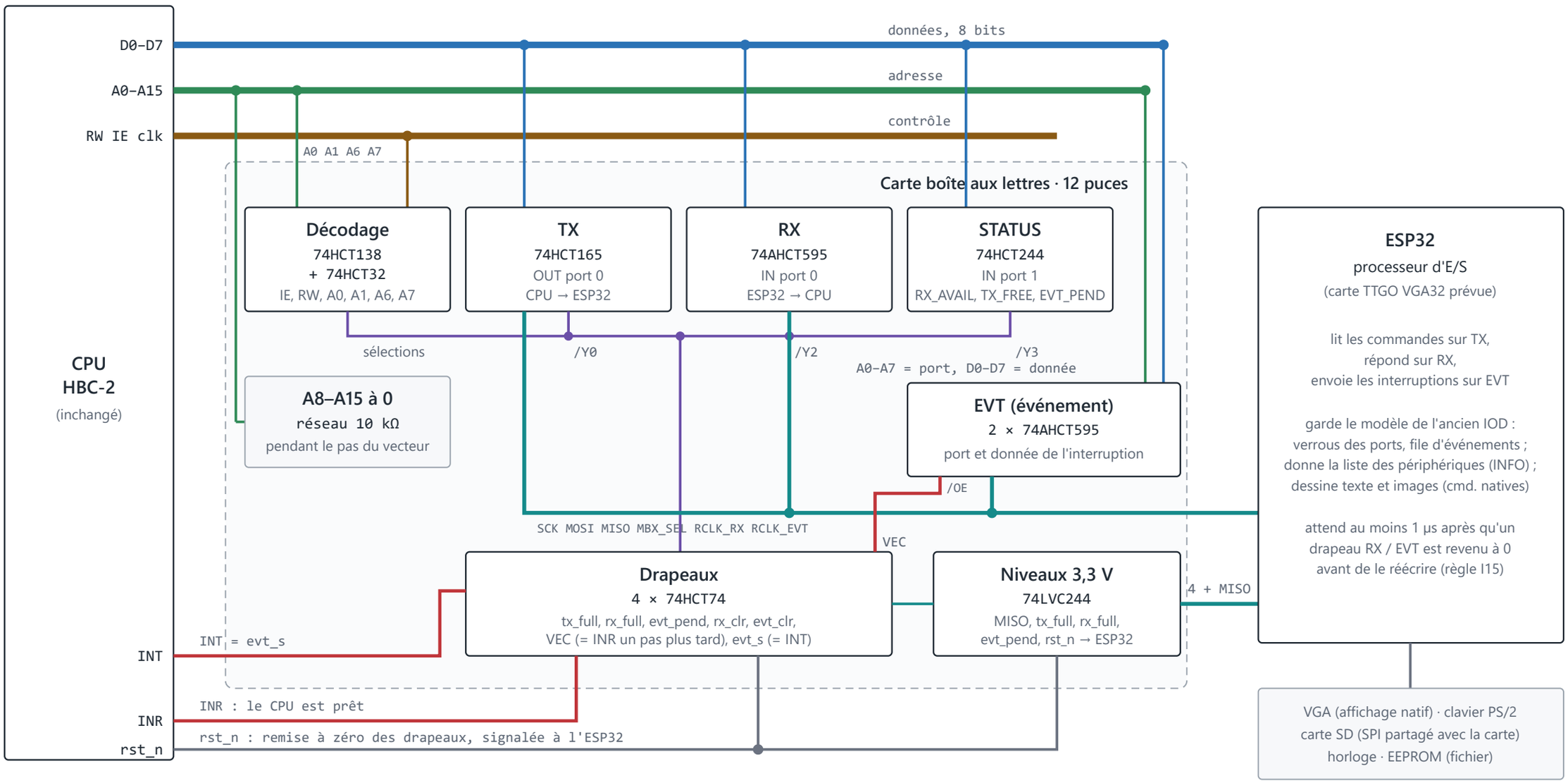
Entrées/sorties du HBC-2 : boîte aux lettres et processeur d'E/S

Schéma bloc simplifié : le CPU garde IN/OUT et l'échange INT/INR. L'IOD devient une carte « boîte aux lettres » de 12 puces devant un ESP32, qui gère les périphériques comme le faisait l'IOD. La RAM et la ROM ne sont pas dessinées.



Carte des ports

PORTS	IN	OUT
`0x00`	RX : l'octet de l'ESP32	TX : un octet pour l'ESP32
`0x01`	STATUS	rien
`0x04, 0x08 … 0x3C` `0x05, 0x09 … 0x3D`	copies de 0x00 et 0x01 (A2–A5 ne sont pas décodés)	
`0x02, 0x03 (mod 4)` `dans 0x00–0x3F`	seconde sélection libre (/Y4–/Y7), non câblée	
`0x40–0xFF`	vides : rien ne répond (0xFF dans l'émulateur)	

STATUS	NOM	À 1 QUAND
`bit 0`	`RX_AVAIL`	un octet attend dans RX
`bit 1`	`TX_FREE`	TX est vide : on peut écrire
`bit 2`	`EVT_PEND`	un événement attend dans EVT

Liste des puces

PUCE	QTÉ	RÔLE
`74HCT138`	1	Décodage : G1 = IE, /G2A = A7, /G2B = A6, A = A0, B = RW, C = A1
`74HCT32`	1	Impulsion d'écriture de TX = OU(/Y0, clk_wr_n) (3 portes libres)
`74HCT74`	4	Drapeaux TX, RX, EVT ; rx_clr, evt_clr, VEC, evt_s = INT (1 bascule libre)
`SN74AHCT595N`	3	RX → données ; port de l'événement → A0–A7 ; donnée de l'événement → données
`CD74HCT165E`	1	TX : chargé depuis le bus de données, lu en série par l'ESP32
`74HCT244`	1	STATUS → bus de données
`SN74LVC244AN`	1	5 V → 3,3 V vers l'ESP32 : MISO (validé par MBX_SEL), tx_full, rx_full, evt_pend, rst_n
`réseau 10 kΩ`	1	Tire A8–A15 à 0 pendant le pas du vecteur (passif, hors des 12 puces)

Commandes (CPU → ESP32)

OCTETS	COMMANDE	RÉPONSE SUR RX
`0x00`	NOP	—
`0x01 p v`	OUT : port p = v	—
`0x02 p`	IN : lit le port p	1 octet
`0x03`	INFO	protocole (2), N, N × 6 octets (classe, produit, port, nb)
`0x10–0x7F`	commandes natives : texte, graphismes, VSYNC, heure (page 3)	les lectures seulement

> Les ports p des périphériques sont dans l'ESP32, plus sur le bus. Au démarrage, le CPU envoie INFO et attend 2 s au plus (l'ESP32 démarre en 1 s environ) ; sans réponse, aucun périphérique. Commandes envoyées interruptions coupées (`mbx_out`, `mbx_in`).

Envoi d'un octet (OUT 0x00)

1. Le CPU lit STATUS jusqu'à TX_FREE = 1.
2. `out` sur le port 0 : le 74HCT165 charge l'octet, `tx_full` passe à 1.
3. L'ESP32 voit `tx_full`, baisse MBX_SEL, lit 8 bits (SCK, MISO), puis remonte MBX_SEL : `tx_full` revient à 0.

Réception d'un octet (IN 0x00)

1. L'ESP32 décale 8 bits dans le 595 RX (SCK, MOSI) et envoie une impulsion sur RCLK_RX : `rx_full` = 1.
2. Le CPU lit STATUS jusqu'à RX_AVAIL = 1, puis fait `in` sur le port 0.
3. `rx_full` revient à 0 au pas suivant.

Déroulement d'une interruption

1. L'ESP32 décale 16 bits dans les 595 EVT (la donnée, puis le port) et envoie une impulsion sur RCLK_EVT : `evt_pend` = 1.
2. Au front d'horloge suivant, `evt_s` recopie `evt_pend` : c'est INT.
3. Si le drapeau I est à 1, le CPU termine son instruction et monte INR.
4. Un pas plus tard (VEC), les 595 EVT mettent le port p sur A0–A7 et la donnée d sur D0–D7 (A8–A15 tirés à 0). Le CPU met d dans I et saute au vecteur `$0xFE00 + 2 × p`, comme avant.
5. `evt_pend` revient à 0 au pas suivant. L'ESP32 attend au moins 1 µs avant l'événement suivant.

Commandes natives (protocole 2)

L'ESP32 dessine lui-même : une image de 256 × 192 pixels, 16 couleurs, du texte en cellules de 8 × 8 (32 colonnes × 24 lignes, caractères Windows-1252). Une commande = un octet de commande et ses arguments ; seules les lectures répondent sur RX. Coût CPU : environ 26 µs par octet (mesuré dans l'émulateur).

Texte (0x10–0x2F)

OCTETS	COMMANDE	RÉPONSE
`0x10 c`	PUTC : caractère ou code de contrôle au curseur	—
`0x11 n` $c_1 \ldots c_n$	PUTS : n caractères (0–255)	—
`0x12 x y`	CURSOR : colonne 0–31, ligne 0–23	—
`0x13 fg bg`	COLOR : couleurs du texte	—
`0x14 bg`	CLEAR : tout l'écran, curseur en 0, 0	—
`0x15 n`	SCROLL : monte de n lignes	—
`0x16 code` $r_0 \ldots r_7$	DEFCHAR : dessin d'un caractère (0x20–0xFF)	—
`0x17 x y`	GET_CHAR	caractère, couleurs
`0x18`	GET_CURSOR	x (32 : retour différé), y
`0x19 on`	SHOW_CURSOR : curseur clignotant	—
`0x1F`	TEXT_RESET : police, couleurs, curseur, écran bleu	—

> Codes de contrôle : `\b` recule, `\t` colonne multiple de 8, `\n` ligne suivante, `\r` début de ligne. Retour à la ligne différé : écrire la dernière cellule ne fait pas défiler, le caractère suivant si. Couleurs au démarrage : jaune vif (14) sur bleu (1).

Graphismes (0x30–0x4F)

OCTETS	COMMANDE	RÉPONSE
`0x30 x y c`	PIXEL	—
`0x31` x_0 y_0 x_1 y_1 `c`	LINE : les deux extrémités	—
`0x32 x y w h c`	RECT : contour (taille 0 = 256)	—
`0x33 x y w h c`	FILL_RECT : rempli (taille 0 = 256)	—
`0x34 x y`	GET_PIXEL	couleur
`0x40`	BLIT (réservé)	

Temps (0x60–0x6F) et réservés

OCTETS	COMMANDE	RÉPONSE
`0x60`	GET_TIME	année − 1900, mois, jour, h, min, s
`0x61 on`	VSYNC : un événement par image (60 Hz), compteur d'images en donnée	—
`0x50–0x5F`	sprites (réservé)	
`0x70–0x7F`	SD_READ, SD_WRITE (réservé)	

> L'affichage natif a un descripteur : classe 03 80, produit 0x0006, 1 port (celui de l'événement VSYNC). Les 5 ports de l'ancien moniteur (03 00) donnent accès à la même image.